IMAGINATION INVENTORS

AmViSH 2.0

Design and Implementation of a Speech-Driven Mobile Robot with Language-Model-Assisted Interaction

PROJECT THESIS

Team Members
Aman Vishwas
Ujjwal Kant
Aditya Kumar
Ekansh Suman

Source-based implementation report
October 2026

Contents

Scope and evidence statement
Abstract
Keywords
Introduction
Related Work and Concepts
System Architecture
  Implemented data flow
  Startup discovery
  Requirements evidenced in source
Implementation Analysis
Audio acquisition and gating
  Speech recognition
  Language-model behavior and context
Transport and bridge
Arduino actuator controller
Action compatibility
Scheduling and speech output
  Alternate sketch and unused peripherals
Review Method and Evidence
Verification Plan
Safety, Security, and Privacy
Discussion
Limitations
Recommendations
Conclusion
Command protocol
Files reviewed
References

Scope and evidence statement

This thesis describes the implementation represented by the accompanying source tree. It is an engineering report, not evidence of a validated autonomous or safety-certified robot. Findings are based on source inspection unless identified as measured results. The repository contains no benchmark dataset, timing logs, controlled experimental record, user study, or independent safety assessment; therefore no numerical claims about accuracy, latency, robustness, or usability are made.

Abstract

AmViSH 2.0 is a prototype mobile-robot interaction system connecting a laptop speech pipeline to microcontroller-based actuators. The laptop captures mono audio in fixed-duration chunks, applies an RMS-energy comparison, transcribes selected chunks locally with OpenAI Whisper, and sends recognized text to a Groq-hosted chat-completion model. The model is prompted for JSON containing a discrete action label and spoken response. The laptop validates the action against an allowlist, posts it to an ESP32 over local HTTP, and speaks the response with pyttsx3. UDP ping/pong discovery locates the ESP32, which forwards the action over UART to an Arduino for motor and servo control. Source analysis identifies important limitations: processing is synchronous; audio gating is a basic heuristic; response validation is partial; several prompted actions are unsupported by firmware; movement has no independent timeout; and the HTTP control endpoint is unauthenticated. The checked-in runtime does not implement camera perception, face recognition, obstacle avoidance, persistent memory, or measured evaluation. This report documents the actual architecture, implementation, safety and privacy implications, limitations, and a verification roadmap.

Keywords

Human-robot interaction; speech interface; Whisper; language model; ESP32; Arduino; embedded control; prototype safety

Introduction

Robotic speech interaction combines audio acquisition, speech recognition, intent interpretation, action selection, command transport, physical actuation, and response generation. A language model can map varied wording to a small action vocabulary, but it does not replace sensing, motor control, or safety engineering. AmViSH 2.0 explores this layered arrangement using laptop software, an ESP32 bridge, and an Arduino controller. The project objectives represented in code are: record speech, transcribe it locally, request a constrained action and utterance from a hosted model, discover the ESP32 without a fixed address, send the action to embedded hardware, and speak the response. This thesis reconstructs the real code path, compares model actions with firmware support, and identifies improvements. It does not claim learning, self-awareness, autonomous goal generation, novel speech recognition, or experimentally demonstrated performance.

Related Work and Concepts

Whisper is a general-purpose speech recognition model trained with large-scale weakly supervised audio and text data. Its use here does not establish recognition accuracy for this robot's microphone or environment [1]. Surveys discuss language models for robotic communication, planning, and control, while identifying challenges in grounding and reliability [2,3]. AmViSH assigns the model a narrower text-to-action role; it does not receive images or raw sensor streams and does not generate motor-level signals. The laptop/ESP32/Arduino split is a layered control architecture. It clarifies component roles but creates failure boundaries across audio, inference, Wi-Fi, HTTP, UART, parsing, and actuation. A syntactically valid command is not necessarily safe in the physical context.

System Architecture

Implemented data flow

Microphone capture at 16 kHz in nominal 3.5-second chunks is followed by RMS gating and local Whisper transcription in English. Non-empty text is sent to Groq with recent process-local conversation entries. The model is asked to return JSON fields ACT and SAY. The host parses JSON, replaces malformed responses or disallowed ACT labels with IDLE, posts the action to ESP32 /ACT, and speaks SAY locally. ESP32 discovery uses `AMVISH_PING` and `AMVISH_PONG` on UDP port 4210. The bridge sends `ACT:` followed by newline over UART at 9600 baud to Arduino.

Startup discovery

The laptop broadcasts for up to ten seconds and uses the IP address of the first matching responder. The responder is not authenticated, an interface is not explicitly selected, and discovery is not retried during the program loop. Failure raises an error and prevents operation.

Requirements evidenced in source

p0.12p0.78 ID	Implementation evidence
F1	Mono audio capture, 16 kHz, nominal 3.5-second chunks (`config.py`, `audio_listener.py`).
F2	RMS-based candidate chunk selection (`audio_listener.py`).
F3	Local Whisper transcription forced to English (`stt_whisper.py`).
F4	Groq completion prompted for ACT and SAY JSON (`brain_llm.py`).
F5	Malformed JSON and non-allowlisted ACT fall back to IDLE (`brain_llm.py`).
F6	UDP discovery and HTTP POST /ACT (`reverse_discovery.py`, `main.py`).
F7	ESP32-to-Arduino UART forwarding and motor/servo dispatch (firmware).

p0.12p0.78 ID	Implementation evidence
F8	Local speech synthesis using pyttsx3 (`tts_engine.py`).

Implementation Analysis

Audio acquisition and gating

sounddevice records fixed-duration chunks and the code computes root-mean-square amplitude. The first chunk initializes `ambient_energy_avg` and is discarded. Later chunks below half that stored estimate update it with smoothing coefficient `ALPHA`; chunks above the estimate multiplied by `MARGIN` are returned. The default `MARGIN` is 0.5, so this is a simple relative amplitude heuristic, not a conventional threshold expressed as ambient level plus margin. There is no frequency analysis, hysteresis, calibration procedure, or trained voice-activity detector. Performance under noise has not been measured.

Speech recognition

Whisper is loaded once when the module initializes. CUDA is used if requested and available, otherwise CPU. Audio is converted to float32, flattened, clipped, and transcribed with English forced. Text is rejected only if shorter than two characters or without alphabetic characters. No confidence score is checked. Hardware-dependent runtime, recognition quality, and model download behavior are not benchmarked in the repository.

Language-model behavior and context

The Groq API key is loaded from the environment. Up to the last eight user/model entries are added to each request; this process-local context is lost at exit and is not persistent memory or learning. The call is synchronous and uses fixed temperature 0.4. JSON parse errors and unknown action labels fall back to IDLE. SAY type, length, and content are not fully validated. An allowlist restricts action syntax but cannot guarantee that a permitted movement is appropriate. The prompt is not a defense against prompt injection.

Transport and bridge

The host posts a manually assembled JSON body to `http:///ACT` with a one-second timeout. Errors are printed and swallowed; the HTTP status is printed but not used as an acknowledgment. The active sketch is `firmware/esp32_cam_act_bridge/esp32_cam_act_bridge.ino`. It joins Wi-Fi with local `secrets.h`, answers UDP discovery, serves HTTP on port 80, minimally extracts the act value, and forwards it on UART using ESP32 GPIO 15 as TX. HTTP has no authentication, encryption, or rate limiting, and parsing is not performed with a JSON library.

Arduino actuator controller

Arduino reads USB serial and SoftwareSerial RX on D2, accepts lines beginning ACT:, and maps recognized strings to states. Motor direction pins are D7, D4, D8, and D12; enable pins are D5 and D6. Three servos use D9, D10, and D11. Forward/backward PWM is 150 and turning PWM is 120. Motors continue running until a later state transition; there is no independent timeout or emergency-stop input. Repeating the current state may not execute again because the main loop only dispatches when nextAct differs from currentAct. Wave gestures use delay calls, blocking serial processing during each gesture.

Action compatibility

p0.34p0.5 Model action	Arduino behavior
`MOVE_FORWARD`, `MOVE_BACKWARD`, `TURN_LEFT`, `TURN_RIGHT`	Implemented as continuous motor drive.
`STOP_ALL`	Maps to idle and disables motor outputs.
`LOOK_UP`, `LOOK_DOWN`, `CENTER_HEAD`	Sets head servo to a preset position.
`WAVE_RIGHT_HAND`, `WAVE_LEFT_HAND`	Three servo swings with blocking delays.
`ATTEND_USER`	Centers head and hands; it does not detect a person.
`NOD_HEAD`, `SHAKE_HEAD`	Not parsed; falls back to idle.
`IDLE`, `NULL`	Not explicitly parsed; unknown values fall back to idle.

This mismatch should be resolved by keeping host and firmware action maps synchronized and testing every action end-to-end.

Scheduling and speech output

The Python loop runs capture, transcription, hosted inference, HTTP, and text-to-speech sequentially. Each stage blocks later stages. pyttsx3 uses rate 170, volume 1.0, and a lock. The program therefore does not implement a non-blocking real-time loop or a defined end-to-end response deadline.

Alternate sketch and unused peripherals

`firmware/ESP32_main/ESP32_main.ino` provides status and UDP discovery but no /ACT endpoint; it cannot replace the documented bridge. Camera/face-event logic is unused. The wiring workbook lists an ultrasonic sensor and I2C display, but the Arduino program does not implement either. Workbook ultrasonic TRIG on D2 conflicts with serial RX on D2 and must not be wired that way without firmware and pin-map changes.

Review Method and Evidence

This is a source-based review of laptop Python modules, three firmware sketches, README, dependency list, pinout, wiring workbook, and previous thesis PDF. Descriptions were checked against control flow, constants, parsers, and pin use. No physical test is claimed. Code facts, expected behavior, and measured results are distinct evidence classes; the repository currently supplies no measured-result dataset.

Verification Plan

Future trials should record revision, board and driver part numbers, operating system, microphone, room noise, network layout, model identifier, and test date. For each utterance, record expected action, transcript, model output, HTTP response, UART message, actuator result, and timestamps. Do not publish credentials or personal recordings.

p0.18p0.48p0.25 Area	Test proposal	Measures
Audio gate	Quiet, speech, fan, music, changing noise; vary settings	False accepts/rejects; estimate evolution
Transcription	Labeled utterances at distances and noise levels	Word error rate; inference time
Model contract	Bad JSON, unknown action, invalid/large SAY, timeout	Schema pass rate; fallback behavior
Discovery	Missing/multiple devices, blocked UDP, Wi-Fi loss	Discovery rate and time
Firmware	Each action, repeated action, unknown or truncated lines	Parsed state and physical output
Safety	Raised wheels, supervised low-speed trials, connection loss	Stop latency and movement duration
Privacy	Inspect capture, provider payload, logs, retention	Data-flow and retention record

Safety, Security, and Privacy

This prototype controls actuators but has no safety-rated controller, obstacle detection, authenticated command channel, encrypted local transport, or motor timeout. Initial tests should use raised wheels, direct supervision, a clear area, and a reachable power disconnect. These precautions do not imply certification. Recognized speech text is sent to a hosted model provider. Operators should be informed and should avoid speaking sensitive information. Keep API keys in ignored environment files and Wi-Fi credentials in ignored local headers. Because the HTTP endpoint is unauthenticated, keep it on a trusted private network and never port-forward it or expose it publicly. Public issue reports should omit credentials, private audio, and personal data.

Discussion

The architecture's useful feature is an inspectable symbolic boundary between a probabilistic language model and deterministic actuator code. The action allowlist, HTTP bridge, UART framing, and Arduino parser provide places to add stronger checks. However, the current implementation has no implemented environmental sensing in the active path to assess movement context. The previous thesis described face detection, dynamic temperature, confidence-based transcription filtering, non-blocking execution, robust fault isolation, and measured stability. These capabilities are not present in the current source. The actual runtime is speech-only and sequential, and its audio gate is a basic amplitude heuristic. These should be described as future work rather than completed results. Motion lifecycle is the most urgent engineering gap. If a movement command reaches Arduino and later communication is lost, motion can continue because there is no actuator-side deadline. A safer design should add a command lease/watchdog and independent physical stop before expanding autonomous behavior.

Limitations

Exact board models, motor driver, power system, and assembly conditions are incompletely documented. Performance depends on host hardware, network, provider, microphone, and model version. The source has heuristic speech gating, partial output validation, unauthenticated HTTP, no message-integrity or acknowledgment protocol, unbounded movement, and unsupported prompted actions. Camera perception, face recognition, obstacle sensing, display output, battery monitoring, persistent memory, and performance measurements are absent. Dependencies are not pinned, reducing reproducibility.

Recommendations

- Add firmware movement timeout/watchdog and independent physical emergency stop.
- Synchronize and test the host allowlist, bridge parser, and Arduino action map.
- Use robust JSON parsing and validate/bound every response field, especially SAY.
- Add command acknowledgments and explicit communication-failure states.
- Calibrate speech gating or use a documented VAD with hysteresis and tests.
- Add bounded retries, timeouts, recovery states, and non-sensitive structured logs.
- Isolate the network and add command authorization.
- Pin tested Python and firmware toolchain dependencies; document exact hardware.
- Add unit, protocol, and hardware-in-loop tests and publish measured results with setup details.
- Claim camera, obstacle sensing, or adaptive behavior only when implemented and evaluated.

Conclusion

AmViSH 2.0 is a prototype voice-interaction system joining local audio capture and speech recognition, hosted language-model interpretation, network transport, and embedded actuation. Its contribution is an integrated, inspectable pipeline and a set of concrete system-integration lessons. The actual source is narrower than earlier narrative claims: processing blocks, perception is speech-only, the audio gate is heuristic, output validation is partial, some labels are unsupported, and movement has no bounded stop. No empirical performance or safety result follows from the supplied files. The next stage should prioritize actuator safety, acknowledgment, robust validation, reproducible builds, and measured evaluation.

Appendix

Command protocol

The host posts JSON such as `act = TURN_LEFT` to `/ACT`. The ESP32 forwards a newline-terminated `ACT:TURN_LEFT` message. Arduino accepts the `ACT:` prefix and maps recognized strings. Message identifiers, action duration, authentication, and acknowledgments are recommendations, not implemented fields.

Files reviewed

Laptop: `main.py`, `config.py`, `audio_listener.py`, `stt_whisper.py`, `brain_llm.py`, `tts_engine.py`, `reverse_discovery.py`. Firmware: `esp32_cam_act_bridge.ino`, `ESP32_main.ino`, `ARDUINO_CODE.ino`. Supporting sources: README, requirements, pinout, workbook, previous `Thesis-original.pdf`.

References

[1] A. Radford et al., Robust Speech Recognition via Large-Scale Weak Supervision, arXiv:2212.04356, 2022. https://arxiv.org/abs/2212.04356.

[2] Y. Kim et al., A Survey on Integration of Large Language Models with Intelligent Robots, arXiv:2404.09228, 2024. https://arxiv.org/abs/2404.09228.

[3] Y. Ma et al., A Survey on Vision-Language-Action Models for Embodied AI, arXiv:2405.14093, 2024. https://arxiv.org/abs/2405.14093.